REPORT PROJECT 1

Part 1 - knapsack problem :

Task 1 : How do you represent a solution to the problem ?

A possible solution to the BKP problem is as follows:

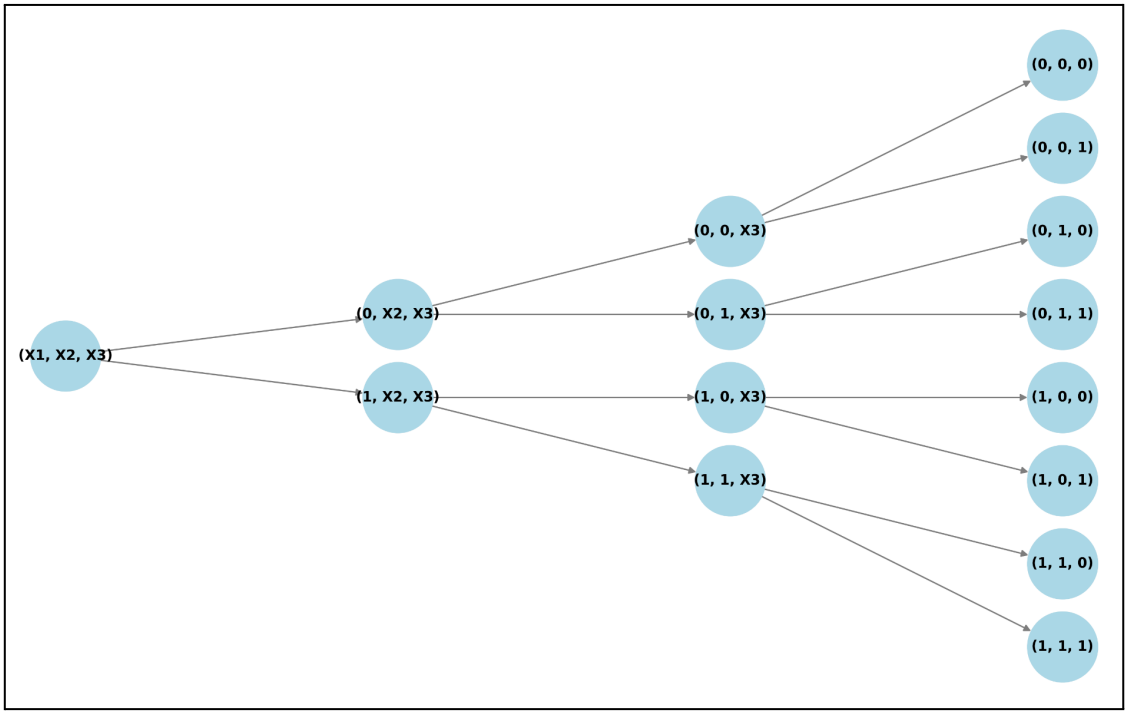
$$X = \{X_1, X_2, \dots, X_i, X_j, \dots, X_n\}$$

where n is the number of objects, and

$$X_i \in \{0, 1\}$$

represent whether we put the i-th item in the backpack : 1 if included, 0 if not.

A solution to the problem is represented as a subset of items that maximize the total value while respecting the weight constraint. This can be represented as a binary decision vector where each element corresponds to an item.



Task 2 : what do we want to maximize ?

The objective function to maximize is :

$$V = \sum_{i=1}^{n} v_i x_i$$

where:

- v_i is the value of item
- x_i is a binary decision variable (1 if the item is included, 0 if it's not)

Task 3 : What is the equation for the restriction(s) of the problem ?

The problem has the following constraint :

$$C = \{\sum_{i=1}^{n} w_i x_i \leq Capacity\}$$

where w_i stands for the weight of the i-th item.

Task 4 : What is the branching factor (b) and the maximum depth (m) ?

Each item can either be included in the backpack or not be included, so the branching factor in the search tree is **2**.

The maximum depth is equal to the total number of items : n, since all the items will be considered in the worst case.

Task 5 : What is the time expended by the algorithms ? How do they compare ?

- The time complexity for **BFS** is **O(**2^n**)** in the worst case since it explores all nodes at a given level of the tree before moving deeper.
- The time complexity for **DFS** is **O(**2^n**)** in the worst case since it checks one branch until the end of it, before backtracking to other branches.

Task 6 : What is the space used in memory by the algorithms and How do they compare ?

- **BFS** has to keep in memory multiple branches for the level it's in, and the branching factor being 2, the space complexity is **O(**2^n**)**.
- Since **DFS** has to only keep the actual path it's in, the space complexity is **O(n).**

Part 2 - Shortest Path from A to B :

Task 1 : How do you represent a solution to the problem ?

A possible solution to the problem is as follows:

$$X = \{X_1, X_2, \dots, X_i, X_j, \dots, X_n\}$$

where n is the number of all the visited cities in the solution, and

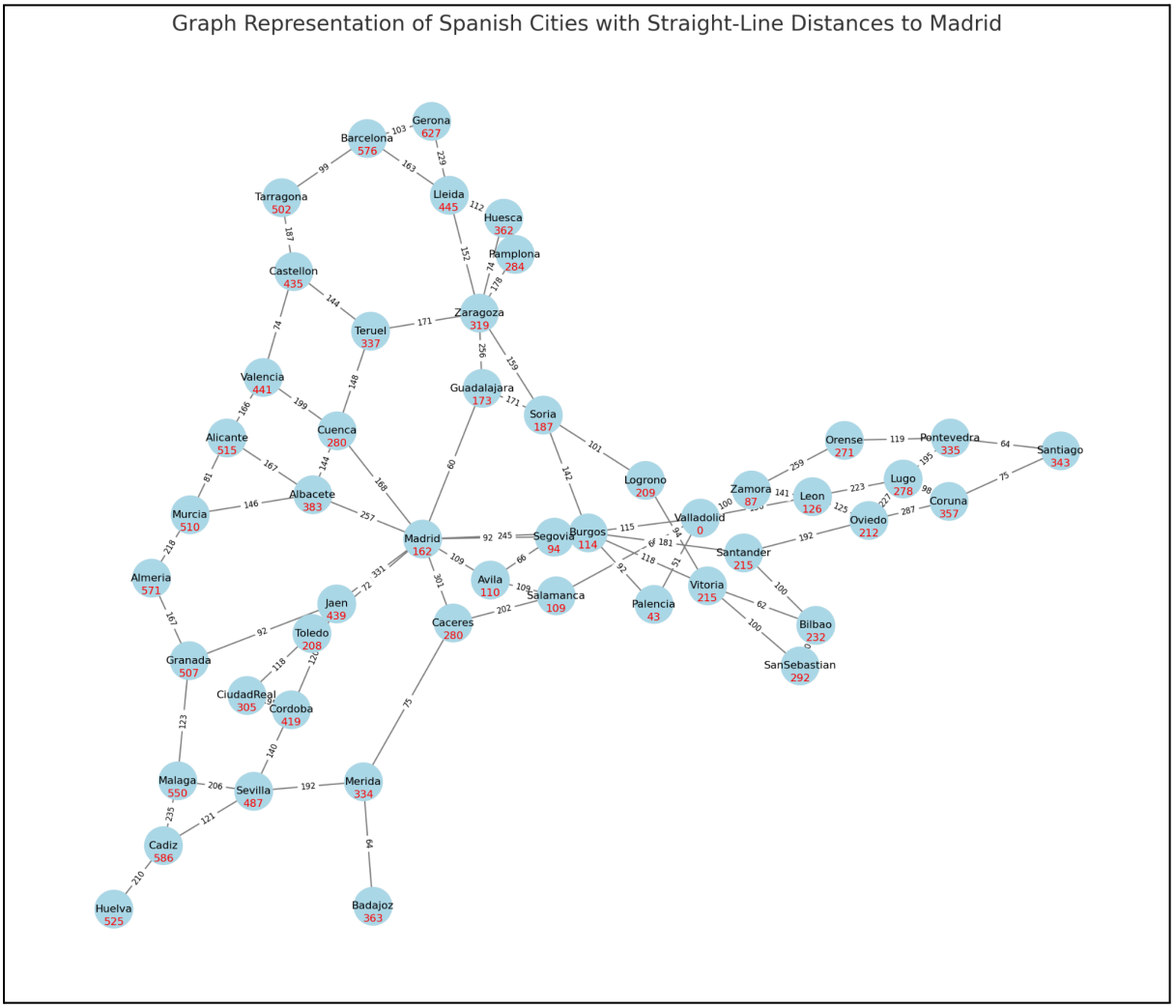
$$X_i, X_j \in \{\,'Almeria', 'Granada', 'Murcia', \dots\,\}$$
$$with\ X_1 = 'Malaga'\ and\ X_n = 'Valladolid'$$

where i stands for the i-th city visited.

A solution is represented as an ordered sequence of nodes (cities) forming the shortest path from the start node (Malaga) to the destination node (Valladolid).

Additionally, the problem has the following constraint :

$$C = \{ X_i \neq X_j, \forall\, i,\ j \ and\ i \neq j \}$$



Task 2 : What is the equation of f(n) used in Greedy Best-first Search ?

GBFS uses a heuristic function $f(n)$ to estimate the cost from node n to the goal. The equation is:

$$f(n) = h(n)$$

where $h(n)$ is the heuristic estimate of the remaining straight line distance from n to the goal (Valladolid).

Task 3 : What is the equation of f(n) used in A* Search ?

A* combines the actual distance to reach a node : $g(n)$, with the heuristic straight line estimate to the goal $h(n)$:

$$f(n) = g(n) + h(n)$$

Task 4 : How do the algorithms compare ?

- **GBFS** only compares the heuristic values of the neighboring nodes relative to the currently visited node. without considering the actual cost to reach them. This comparison on a restricted set of nodes makes the algorithm fast but not necessarily correct.
- By also considering the cost between cities, **A*** can guarantee a correct solution, but needs more memory than GBFS and can be slower. Another difference is that when choosing the next city to visit, the algorithm compares between **all** the leaf nodes of the search tree which means it can backtrack to previously skipped branches if they lead to a better solution.